به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین چهارم

مدرس: مهدی صفرنژاد — انتشار: ۲۱ فروردین ۹۸

درهم‌سازی

سؤال ۱. [۱۵ نمره] یک مجموعه داده $X = \{x_۱, x_۲, \ldots, x_n\}$ داریم. در هر کدام از زیرمسئله‌های زیر با توجه به تابع درهم سازی $h(x)$ و مقادیر x_iهای تعریف شده (که مجموعهٔ X را تشکیل می‌دهند) نسبت طولانی ترین زنجیره تشکیل شده به تعداد کل داده‌ها را بیابید. در هر قسمت به صورت کوتاه توضیح دهید. ٪ عملگر باقی‌مانده است.

آ.

$$x_i = i \,\%\, ۱۰۵ \quad h(x) = x \,\%\, ۱۰۰$$

ب.

$$x_i = ۴ \times i + ۳ \quad h(x) = x \,\%\, ۹۰$$

پ.

$$x_i = (۳ \times i) \,\%\, ۱۰۰ \quad h(x) = \lfloor \sqrt{x} \rfloor \,\%\, ۹$$

ت.

$$x_i = i^۲ \quad h(x) = x \,\%\, ۱۰$$

ث.

$$x_i = i^i \quad h(x) = x \,\%\, ۱۰$$

پاسخ: نسبت را برای nهای بزرگ در نظر می‌گیریم پس با توجه به این برای هر بخش خواهیم داشت

آ.

$$X = \{۱, ۲, ۳, ۴, \ldots, ۱۰۳, ۱۰۴, ۰, ۱, ۲, \ldots\},\ h(X) = \{۱, ۲, ۳, \ldots, ۹۹, ۰, ۱, ۲, ۳, ۴, ۰ \ldots\} \to \frac{۲}{۱۰۵}$$

بعد از هر ۱۰۵ بار $h(X)$ تکرار می‌شود و حداکثر دو مقدار تکراری در هر دوره دارد.

ب.

$$X = \{۷, ۱۱, ۱۵, ۱۹, \ldots\},\ h(X) = \{۷, ۱۱, \ldots, ۸۴, ۱, ۵, ۹, \ldots, ۳, ۷, ۱۱, \ldots\} \to \frac{۱}{۴۵}$$

بعد از هر ۴۵ بار $h(X)$ تکرار می‌شود و حداکثر یک مقدار تکراری در هر دوره دارد.

پ.

$$X = \{۳, ۶, ۹, ۱۲, \ldots, ۹۹, ۲, ۵, ۸, ۱۱, \ldots\ ۹۸, ۱, ۴, ۷, \ldots, ۹۷, ۰, ۳, \ldots\},$$

$$h(X) = \{۱, ۲, ۳, ۳ \dots, ۰, ۱, ۲, ۲, ۳ \dots, ۰, ۱, ۲, ۲\} \to \frac{۱}{۵}$$

بعد از هر ۱۰۰ بار $h(X)$ تکرار می‌شود و حداکثر بیست مقدار تکراری (به تعداد صفرها) در هر دوره دارد.

ت.

$$X = \{۱, ۴, ۹, ۱۶, \dots\},\ h(X) = \{۱, ۴, ۹, ۶, ۵, ۶, ۹, ۴, ۰, ۱, \dots\} \to \frac{۱}{۵}$$

بعد از هر ۱۰ بار $h(X)$ تکرار می‌شود و حداکثر دو مقدار تکراری در هر دوره دارد.

ث.

$$X = \{۱, ۴, ۲۷, ۶۴, \dots\},\ h(X) = \{۱, ۴, ۷, ۴, ۵, ۶, ۳, ۴, ۹, ۰, ۱ \dots\} \to \frac{۳}{۱۰}$$

بعد از هر ۱۰ بار $h(X)$ تکرار می‌شود و حداکثر سه مقدار تکراری در هر دوره دارد.

سؤال ۲. **[۱۵ نمره]** فرض کنید یک جدول آدرس دهی باز داریم که به صورت زیر با استفاده از وارسی خطی پر شده است (تابع درهمسازی برابر است با $h(x) = x \,٪\, ۹$).

۳۶	۹	۱۱	۱۸	۴۰	۱۳	۲۴	۳۱	۷۷

آ. **[۶ نمره]** ترتیب اولیه اعداد وارد شده، چند حالت می تواند داشته باشد؟ به صورت کوتاه توضیح دهید.

ب. **[۳ نمره]** برای پاک کردن عدد ۱۸ چه خانه هایی باید وارسی شوند؟

پ. **[۶ نمره]** فرض کنید جدول درهم سازی به طول n با تابع در همسازی $h(x)$ ($h(x)$ مشخص نیست) و اعداد $a_۱, a_۲, ..., a_n$ داریم. آیا می توانیم این جدول را با تعداد محدودی عمل درج و حذف به جدولی با اعداد $b_۱, b_۲, ...b_n$ تبدیل کنیم به طوری که $b_۱, b_۲, ..., b_n$ یک جایگشت از $a_۱, a_۲, ..., a_n$ است؟ (هر کدام از a_i ها و b_i ها می توانند خالی باشند و از وارسی خطی برای درج و حذف استفاده می کنیم)

پاسخ: آ. با چند حالت‌بندی ساده نتیجه می‌شود: $\binom{۹}{۴} \cdot ۹ = ۱۱۳۴$

ب. باید خانه‌های صفرم تا سوم را چک کنیم.

پ. خیر نمی‌شود. فرض کنید جدول بالا را با اعداد ۹, ۱۸, ۲۷, ۳۶, ۴۵, ۵۴, ۶۳, ۷۲ داریم (خانهٔ آخر خالی است). به هیچ وجه نمی‌توان اعداد داخل این جدول را یک واحد به جلو شیفت داد (به طوری که خانهٔ اول خالی باشد و بعد از آن اعداد به همان ترتیب بالا بیایند).

سؤال ۳. **[۲۰ نمره]** فرض کنید H یک خانواده از توابع درهم‌سازی باشد که هر تابع درهم‌سازی $h \in H$ مجموعهٔ U از کلیدها را به $\{۰, ۱, ..., m-۱\}$ می‌برد. می‌گوییم خانوادهٔ H از توابع درهم‌سازی k-universal است اگر برای هر kتایی متمایز $(x_۱, x_۲, ..., x_k) \in U^k$ و هر k تایی $(y_۱, y_۲, ..., y_k) \in [m]^k$ و هر $h \in H$ که به صورت تصادفی انتخاب شده باشد داشته باشیم:

$$Pr(h(x_۱) = y_۱ \wedge h(x_۲) = y_۲ \wedge ... \wedge h(x_k) = y_k) = m^{-k}.$$

به عبارت دیگر، k تایی $(h(x_۱), h(x_۲), ..., h(x_n))$ با احتمالی برابر می‌تواند هر کدام از k تایی‌های مرتب از مجموعه‌ی $[m]^k$ باشد.

آ. [۴ نمره] ثابت کنید اگر H یک خانوادهٔ 2-universal باشد، universal نیز هست.

ب. [۶ نمره] فرض کنید U مجموعهٔ تمام nتایی‌ها از $Z_p = \{۰, ۱, ..., p-۱\}$ باشد که p یک عدد اول است. کلید $a = (a_۱, a_۲, ..., a_n) \in U$ را در نظر بگیرید. برای هر n تایی $x = (x_۱, x_۲, ..., x_n) \in U$ تابع درهم‌سازی h_a را به این صورت تعریف می‌کنیم

$$h_a(x) = (\sum_{j=۱}^{n} a_j x_j) \bmod p$$

فرض کنید H خانوادهٔ تمام h_a ها باشد. ثابت کنید H یک خانوادهٔ universal است ولی 2-universal نیست.

پ. [۶ نمره] حال خانوادهٔ H در قسمت قبل را کمی تغییر می‌دهیم. برای هر $a \in U$ و هر $b \in Z_p$ تابع h'_{ab} را به این صورت تعریف می‌کنیم

$$h'_{ab}(x) = (\sum_{j=۱}^{n} a_j x_j + b) \bmod p$$

فرض کنید H' خانواده‌ی تمام h'_{ab} ها باشد. ثابت کنید H' یک خانوادهٔ 2-universal است.

ت. [۴ نمره] فرض کنید علی و رضا بین خودشان یک تابع درهم‌سازی h را از خانوادهٔ 2-universal که H نام دارد انتخاب کنند و آن را با هیچ کس در میان نگذارند. هر تابع درهم‌سازی $h \in H$ مجموعهٔ U از کلیدها را به Z_p می‌برد که p یک عدد اول است. علی از طریق اینترنت برای رضا پیام $m \in U$ را می‌فرستد. او برای اطمینان از درست رسیدن پیامش $t = h(m)$ را نیز همراه پیامش می‌فرستد. رضا بعد از دریافت پیام چک می‌کند که آیا رابطه‌ی $t = h(m)$ برقرار است یا نه. فرض کنید فردی در مسیر فرستاده شدن پیام مداخله کند تا پیامی اشتباه به دست رضا برسد. او برای این کار (m, t) را با (m', t') عوض می‌کند تا رضا را فریب دهد. نشان دهید حتی در صورتی که آن فرد خانواده‌ی H را بداند، احتمال این که بتواند رضا را فریب دهد طوری که رضا (m', t') را قبول کند حداکثر $\frac{۱}{p}$ است.

پاسخ:

آ. اگر H یک خانوادهٔ 2-universal باشد آنگاه برای هر دو کلید $x \neq y$ زوج $(h(x), h(y))$ با احتمال یکسانی می‌تواند هر کدام از اعضای مجموعهٔ $\{۰, ۱, ..., m-۱\} \times \{۰, ۱, ..., m-۱\}$ باشد. در m حالت از $m^۲$ حالتی که این زوج می‌تواند داشته باشد برخورد داریم. پس اگر تمام hها را در نظر بگیریم در $\frac{۱}{m}|H|$ از آن‌ها برخورد رخ می‌دهد پس H یک خانواده‌ی universal است.

ب. دو کلید متفاوت x و y را در نظر بگیرید. بدون کاسته شدن از کلیت مسأله فرض کنید این دو کلید در جایگاه اول متفاوت باشند. برخورد در صورتی رخ می‌دهد که داشته باشیم:

$$a_۱(x_۱ - y_۱) + a_۲(x_۲ - y_۲) + ... + a_n(x_n - y_n) \equiv ۰ \pmod p$$

$$\Leftrightarrow a_۱(x_۱ - y_۱) \equiv -(a_۲(x_۲ - y_۲) + ... + a_n(x_n - y_n)) \pmod p$$

حال $a_۲$ تا a_n را ثابت نگاه می‌داریم و $a_۱$ را تغییر می‌دهیم. دقت کنید سمت راست عددی ثابت است. همچنین می‌دانیم که باقیماندهٔ $x_۱ - y_۱$ به p برابر ۰ نیست. پس از p حالتی که می‌توانیم $a_۱$ را مقدارگذاری کنیم دقیقاً یکی وجود دارد که رابطهٔ بالا برقرار شود. پس حداکثر برای $\frac{۱}{p}$ از اعضای H داریم $h_a(x) = h_a(y)$. پس این خانواده universal است. حال کلید $x = (۰, ۰, ..., ۰)$

و یک y ثابت را در نظر بگیرید. برای هر عضو از H داریم $(h_a(x), h_a(y)) = (۰, h_a(y))$. پس $(h_a(x), h_a(y))$ نمی‌تواند با احتمال یکسانی برابر هر کدام از اعضای $\{۰, ۱, ..., p-۱\} \times \{۰, ۱, ..., p-۱\}$ باشد. پس این خانواده 2-universal نیست.

پ. دو کلید متفاوت x و y را در نظر بگیرید و مانند قبل فرض کنید در جایگاه اول متفاوت هستند. $a_۲$ تا a_n را ثابت می‌گیریم. ثابت می‌کنیم برای هر r و t از $\{۰, ۱, ..., p-۱\}$ دقیقا یک زوج $(a_۱, b)$ وجود دارد به طوری که $(h(x), h(y)) = (r, t)$. باید داشته باشیم:

$$r \equiv a_۱ x_۱ + b + X \pmod{p}$$

$$t \equiv a_۱ y_۱ + b + Y \pmod{p}$$

$$\Leftrightarrow$$

$$a_۱ \equiv (r - t - X + Y)(x_۱ - y_۱)^{-۱} \pmod{p}$$

$$b \equiv r - a_۱ x_۱ - X \pmod{p}$$

طبق فرض می‌دانیم که $(x_۱ - y_۱)^{-۱}$ به صورت یکتا وجود دارد. پس حکمی که می‌خواستیم ثابت کنیم ثابت شد. پس برای هر انتخاب $a_۲$ تا a_n هر زوج (r, t) دقیقا یک بار تولید می‌شود. پس هر زوج (r, t) دقیقا $p^{n-۱}$ بار تولید می‌شود. پس این خانواده 2-universal است.

ت. برای کلیدهای m و m' اگر h تصادفی انتخاب شود، زوج $(h(m), h(m'))$ با احتمال یکسانی می‌تواند هر کدام از اعضای $\{۰, ۱, ..., p-۱\} \times \{۰, ۱, ..., p-۱\}$ باشد. پس تمام زوج‌های $(t, h(m'))$ با احتمال یکسان $\frac{۱}{p}$ می‌توانند رخ دهند. پس فردی که می‌خواهد پیام را عوض کند با دانستن H و با دیدن (m, t) هیچ چیزی درباره‌ی $h(m')$ نمی‌فهمد. او به دنبال زوج $(t, h(m'))$ ای است که احتمال رخ دادن آن بیشینه باشد. از آن جایی که تمام این زوج‌ها با احتمال یکسان $\frac{۱}{p}$ رخ می‌دهند، پس او باید یکی از آن‌ها را تصادفی انتخاب کند. در نتیجه احتمال اینکه رضا پیام را قبول کند برابر $\frac{۱}{p}$ خواهد بود.

سؤال ۴. [۱۵ نمره]

درخت T را در نظر بگیرید. مجموعه E از گراف‌ها در اختیار شما قرار میگیرد؛ به طوری که هر عضو از این مجموعه گرافی است که از حذف یک رأس و یال‌های متصل به آن از T تولید می‌شود. با این فرض که تعداد راس های گراف T برابر n باشد. الگوریتمی از مرتبهٔ $O(n^۳)$ ارائه دهید که با دریافت مجموعهٔ E گراف T را تولید کند.

پاسخ: با توجه به اینکه گراف اولیه یک درخت بوده است؛حداقل دو گراف همبند در مجموعه E وجود دارد (یک درخت حداقل دو برگ دارد). برای یافتن این دو گراف کافیست پردازشی از $O(n^۲)$ انجام شود.
یا توجه به اینکه تفاوت این دو درخت تنها در دو برگ است؛ با اتصال یک برگ به درخت اول و یک برگ به درخت دوم $n^۲$ درخت جدید می‌سازیم. هر یک از این درخت‌ها را درهم‌سازی می‌کنیم. حال کافی است درهم‌سازی یک درخت از مجموعه تولید شده توسط درخت اول با درهم‌سازی درختی از مجموعهٔ دوم برابر شود.
عملیات فوق در $O(n^۳)$ انجام پذیر است.

سؤال ۵. [۱۵ نمره] رشته‌ای به طول n داریم. الگوریتمی ارائه دهید که تعداد زیررشته‌های متقارن این رشته را در زمان $\mathcal{O}(n \log n)$ محاسبه کند. برای این سوال یک تقریب مناسب از جواب نیز کفایت می‌کند، به عبارت دیگر در صورت پیاده‌سازی یک تابع درهم‌سازی مناسب نیازی نیست حالاتی که hash برابر است اما رشته‌ها برابر نیستند را بررسی کنید. همچنین منظور از رشتهٔ متقارن رشته‌ای است که از دو طرف به یک شکل خوانده شود مانند abccba و یا aca.

پاسخ: رشته مسئله را s می‌نامیم. برای این سؤال ابتدا درهم‌سازی تمامی زیررشته‌های به فرم $s[۰, t]$ را محاسبه می‌کنیم. برای این کار درهم‌سازی را به صورت زیر در نظر می‌گیریم (برای m و k مناسب)

$$h(s[۰, t]) = \sum_{i=۰}^{t} s[i]k^i \mod m$$

دقت کنید محاسبه تمامی این درهم‌سازی‌ها در زمان $\mathcal{O}(n)$ انجام می‌شود. همین کار را برای وارونه رشته s (که آن را s' می‌نامیم) نیز انجام می‌دهیم. توجه کنید با این کار درهم‌سازی یک زیررشتهٔ خاص در زمان $\mathcal{O}(۱)$ پیدا می‌شود، زیرا

$$h(s[i, j]) = h(s[۰, j]) - h(s[۰, i - ۱])$$

پس درهم‌سازی زیررشتهٔ $s[i, j]$ در زمان $\mathcal{O}(۱)$ محاسبه می‌شود. به طور مشابه درهم‌سازی وارون آن نیز در $\mathcal{O}(۱)$ محاسبه می‌شود، پس می‌توان متقارن بودن آن را در $\mathcal{O}(۱)$ بررسی کرد. پس تا اینجا نشان دادیم که بررسی متقارن بودن هر زیررشته در $\mathcal{O}(۱)$ انجام می‌شود.

ابتدا تعداد زیررشته‌های متقارن با طول فرد را پیدا می‌کنیم. توجه کنید هر یک از این زیررشته‌ها یک حرف وسط دارد. اندیس این حرف وسط را k می‌گیریم. حال توجه کنید که اگر $s[k - i, k + i]$ متقارن باشد؛ تمام زیررشته‌های کوچک‌تر با حرف وسط k نیز متقارن هستند؛ به عبارت دیگر $s[k - i + ۱, k + i - ۱]$ و ...و $s[k - ۱, k + ۱]$ نیز متقارن هستند (می‌توان خود حرف را نیز به عنوان زیررشته متقارن حساب کرد). پس کافی است برای هر حرف وسطی، بزرگترین زیررشته متقارن به مرکز آن حرف را پیدا کنیم. بنابر توضیحات قبلی برای این کار می‌توانیم از جست‌وجوی دودویی استفاده کنیم. با استفاده از جست‌وجوی دودویی کافی است برای هر حرف وسط تنها متقارن بودن $\mathcal{O}(\log n)$ زیررشته را بررسی کنیم، که هرکدام در زمان $\mathcal{O}(۱)$ انجام می‌شود. پس تعداد زیررشته‌های متقارن به مرکز یک حرف خاص در زمان $\mathcal{O}(\log n)$ محاسبه می‌شود. از آن‌جا که در کل n حرف داریم، تعداد کل زیررشته‌های متقارن فرد در زمان $\mathcal{O}(n \log n)$ قابل محاسبه است.

برای راحتی، یک تعریف می‌کنیم. به فاصله بین دو حرف از رشته یک دو ـ فاصله می‌گوییم. در حقیقت مسئله برای زیررشته‌های به طول زوج فرق خاصی با زیررشته‌های به طول فرد ندارد. کافی است توجه کنیم که زیررشته‌های زوج حرف وسط ندارند؛ اما دو ـ فاصله وسط دارند. مشابه جست‌وجوی دودویی را می‌توان در مورد دو ـ فاصله‌ها به کار برد. با استفاده از جست‌وجوی دودویی کافی است برای هر دو ـ فاصله وسط تنها متقارن بودن $\mathcal{O}(\log n)$ زیررشته را بررسی کنیم، که هرکدام در زمان $\mathcal{O}(۱)$ انجام می‌شود. پس تعداد زیررشته‌های متقارن به مرکز یک دو ـ فاصله خاص در زمان $\mathcal{O}(\log n)$ محاسبه می‌شود. از آن‌جا که در کل $n - ۱$ دو ـ فاصله داریم، تعداد کل زیررشته‌های متقارن زوج نیز در زمان $\mathcal{O}(n \log n)$ محاسبه می‌شود. با جمع زدن تمامی زمان‌های فوق می‌توان دید تعداد کل زیررشته‌های متقارن در زمان $\mathcal{O}(n \log n)$ محاسبه می‌شود.

سؤال ۶. [۲۰ نمره] خوب، بد، زشت

دکتر شکیبا از دوران کودکی حروف کوچک انگلیسی را به دو دستهٔ خوب و بد تقسیم می‌کند. او یک رشته از حروف را زیبا می‌نامد، اگر حداکثر k حرف بد در آن یافت شود. به ما یک رشته از حروف کوچک انگلیسی داده‌شده است. قصد داریم تعداد زیررشته‌های زیبای متمایز آن را مطابق با استانداردهای دکتر شکیبا پیدا کنیم.

نکات:

- در این سوال مجاز به استفاده از توابع hash پایتون نیستید.
- در این سوال اجازه استفاده از اپراتور $==$ و ابزارهای مشابه برای مقایسه رشته‌ها را ندارید.

ورودی:

اولین خط ورودی شامل رشته‌ی s است. در خط بعدی یک دنباله‌ی ۲۶ تایی از ۰ و ۱ به شما داده می‌شود که خوب یا بد بودن حروف الفبا را نشان می‌دهد. (خوبی با ۱ و بدی با ۰ نشان داده می‌شود) سپس در خط آخر عدد k به شما داده می‌شود.

$$۱ \leq |s| \leq ۱۵۰۰$$

$$۱ \leq k \leq ۱۵۰۰$$

خروجی:

در تنها خط خروجی باید تعداد زیررشته‌های زیبای متمایز رشته‌ی s را چاپ کنید.

ورودی و خروجی نمونه:

input	output
ababab 01000000000000000000000000 1	5

پاسخ: در هر زیر رشته باید در $O(۱)$ بفهمیم که این زیر رشته زیباست یا نه. برای این کار ابتدا یک پیش‌پردازش از $O(n)$ باید روی تمام زیر رشته ها انجام بدهیم و با استفاده از partial sum تعداد حروف بد را از ابتدا تا مکان x حساب کنیم. سپس برای هر رشته کافی است عدد انتهای رشته را از عدد ابتدای رشته کم کنیم و جواب را به‌دست آوریم حال که می‌توانیم برای هر زیر رشته چک کنیم که زیباست یا نه. در انتها کافی است روی تمام زیر رشته‌ها با $O(n^۲)$ حرکت کنیم و اگر خوب بود درهم‌سازی آن را به مجموعه زیر رشته‌های خوب اضافه کنیم (چون خود آن می‌تواند بسیار بزرگ باشد و در حافظهٔ ما جای نگیرد) همچنین درهم‌سازی هر زیر رشته را می‌توان در $O(۱)$ حساب کرد. کافی است هر کاراکتر که اضافه می‌شود، عدد مربوط به آن را اضافه کنیم و هر کاراکتر که کم می‌شود عدد مربوط به آن را از مقدار درهم‌سازی فعلی کم کنیم و در مواقعی که کاراکتر از سمت چپ کم می‌شود علاوه بر کم کردن درهم‌سازی آن باید مقدار کل درهم‌سازی را تقسیم بر یک عدد نیز بکنیم.

کد آقای وحید زهتاب با نام GBU_sol.py به عنوان نمونه در پوشه مربوطه قرار داده شده.

سؤال ۷. [۲۵ نمره] دگرگونی

فرض کنید که s یک رشته با طول n باشد. برای اعداد صحیح i و j که $0 \leq i < j < n$ تابع f را به صورت زیر تعریف می‌کنیم:

$$f(s,i,j) = s[i+1 : j-1] + r(s[j : n-1]) + r(s[0 : i])$$

که در اینجا منظور از $s[p : q]$ زیررشته‌ای از s است که از p شروع و به q ختم می‌شود. علامت $+$ به معنی ادغام دو رشته است و همچنین تابع r معکوس رشته ورودی را برمی‌گرداند. (اگر $j = i+1$ زیر رشته $s[i+1 : j-1]$ تهی در نظر گرفته می‌شود)

با این تعاریف، به شما دو رشته a و b به عنوان ورودی داده می‌شود و به عنوان خروجی باید i و jای را به عنوان خروجی چاپ کنید که $f(a,i,j) = b$ به طوری که i بزرگترین مقدار ممکن باشد و اگر برای این i چند مقدار j وجود داشت، کم‌ترین مقدار j را انتخاب کنید.

ورودی:
در دو خط اول ورودی، دو رشتهٔ ناتهی a و b با طول کم‌تر از 10^6 به عنوان ورودی داده می‌شوند. (رشته‌ها می‌توانند شامل هر کاراکتری با کد ASCII بین ۳۲ و ۱۲۶ باشند)

خروجی:
دو عدد i و j به عنوان خروجی چاپ می‌شوند. اگر به ازای ورودی جوابی وجود نداشت، عبارت `-1 -1` چاپ می‌شود.

ورودی و خروجی نمونه:

input	output
cbaaaa aaaabc	4 5

input	output
123342 3324212	-1 -1

input	output
abc cba	1 2

پاسخ: رشتهٔ c را معکوس رشتهٔ b در نظر بگیرید. در این صورت باید i و jای را بیابیم که

$$c = a[0 : i] + a[j : n-1] + r(a[i+1 : j-1])$$

برای این کار بیشترین iای که $c[0 : i] = a[0 : i]$ را پیدا می کنیم. سپس به ازای هر i بین ۰ تا این مقدار بیشینه، اگر بیشترین مقدار k را پیدا کنیم که $c[i+1 : k] = a[n-(k-i) : n-1]$ می توانیم چک کنیم که خواستهٔ مسئله برقرار است یا نه (این

که به ازای i فقط بیشترین k کافیست به‌خاطر کمینه بودن j در حالتی که چند جواب برای یک i داشتیم است).

حال برای پیدا کردن چنین k بیشینه‌ای برای هرi، به صورت برعکس عمل می‌کنیم و به ازای هر k کوچک‌ترین iای را می‌یابیم که $c[i+۱ : k] = a[n-(k-i) : n-۱]$ برقرار باشد. برای این کار کافیست روی i، جست‌وجوی دودویی انجام دهیم (چون اگر معادله بالا به ازای یک i برقرار باشد به ازای مقادیر بیشتر از i نیز صادق است، بنابراین می‌توانیم از جست‌وجوی دودویی استفاده کنیم.)

چک کردن این که دو زیر رشته از a و c آیا با هم برابرند یا نه هم را با نگه داشتن درهم‌سازی هر پیشوند از ۰ تا i از رشته‌های a و c می‌توان انجام داد. چون با داشتن این مقادیر درهم‌سازی از i تا j رشته را می توانیم در $O(۱)$ بدست بیاوریم.

بنابراین پیچیدگی زمانی هم از $O(nlog(n))$ می باشد.

کد آقای رضا یاربخش با نام Degarguni_ sol.py به عنوان نمونه در پوشه مربوطه قرار داده شده.

موفق باشید